

Real-Time Order Tracking System – Complete Requirements Document

1. PROJECT OVERVIEW

This system allows customers to place orders online and track the status of their orders in real time. Admins can manage products and update order statuses, which are instantly reflected on the user's dashboard via WebSockets.

2. GOALS & OBJECTIVES

Develop a secure, responsive web application.
Implement real-time tracking using WebSockets or SSE.
Use Spring Boot for scalable backend services.
Use Angular for modern, user-friendly UI.
Maintain clean, modular, testable architecture.

3. FUNCTIONAL REQUIREMENTS



USER MANAGEMENT

Register/Login (JWT-based).
Role-based access: Customer, Admin.
Profile management.



PRODUCT MANAGEMENT (Admin)

Create, update, delete products.
Upload product images.
Manage stock and inventory.



ORDER MANAGEMENT

Customers can place orders.
Admin can update order status (PLACED → PACKED → SHIPPED → DELIVERED).
Cancel/return functionality.
Generate PDF invoices.



REAL-TIME TRACKING

Push notifications via WebSockets or Server-Sent Events (SSE).
Real-time order status changes visible to customers.



ANALYTICS & DASHBOARD

Admin dashboard with stats (total orders, revenue, etc.).
Filter/sort by date, status.
Low-stock alerts.



NOTIFICATIONS

Email confirmation for orders.
In-app notification system.



ADVANCED FEATURES

2FA via email OTP.
Role-Based Access Control (RBAC).
PWA support for mobile.

4. NON-FUNCTIONAL REQUIREMENTS

Security: JWT, Spring Security, password hashing (BCrypt).
Scalability: Microservices-ready architecture.
Performance: Redis or in-memory caching.
Maintainability: Layered architecture (Controller-Service-Repository).

5. SYSTEM ARCHITECTURE

FRONTEND (Angular)

Angular 15+
Angular Material UI
RxJS for state management
Responsive UI
Progressive Web App support

BACKEND (Spring Boot)

Spring Boot 3
Spring Web, Spring Security, Spring Data JPA
WebSocket/STOMP for real-time messaging

DATABASE

PostgreSQL / MySQL
JPA/Hibernate ORM

6. DATABASE DESIGN (ER MODEL)

Entities:

User: id, username, email, passwordHash, role
Product: id, name, description, price, stock
Order: id, user_id, order_date, total_amount, status
OrderItem: id, order_id, product_id, quantity, price

Relationships:

One User has many Orders
One Order has many OrderItems
One Product has many OrderItems

7. WORKFLOW (REAL-TIME TRACKING)

Customer places an order → status = PLACED
Admin processes the order → status = PACKED
Admin ships the order → status = SHIPPED
Order delivered → status = DELIVERED
Customer receives real-time updates
Return/Cancellation (optional) managed by Admin

8. EXTRA FEATURES

Inventory threshold alerts
Swagger/OpenAPI documentation
Role-based permissions
GDPR compliance (data download/delete)

10. API ENDPOINTS

AUTHENTICATION

POST /api/auth/register – Register new user

POST /api/auth/login – Login and receive JWT
POST /api/auth/verify-otp – Verify OTP (2FA)

USER MANAGEMENT

GET /api/users/profile – Get logged-in user's profile
GET /api/users/{id} – Admin fetch user by ID

PRODUCT MANAGEMENT

GET /api/products – Get all products (with filter & pagination)
GET /api/products/{id} – Get product by ID
POST /api/products – Create product (Admin)
PUT /api/products/{id} – Update product
DELETE /api/products/{id} – Delete product
POST /api/products/{id}/upload-image – Upload image file

ORDER MANAGEMENT

POST /api/orders – Place order
GET /api/orders/my – Get orders for current user
GET /api/orders/{id} – Get order by ID

ADMIN ORDER MANAGEMENT

GET /api/admin/orders – Get all orders (Admin)
PUT /api/admin/orders/{id}/status – Update order status

DASHBOARD / STATS

GET /api/admin/stats – Get sales, orders, and user stats
GET /api/admin/inventory/low-stock – View low stock alerts

REAL-TIME WEBSOCKET

WebSocket endpoint: /ws/orders
Subscribe to topic: /topic/orders/{userId}

Message Payload:

```
{  
  "orderId": 101,  
  "status": "SHIPPED",  
  "timestamp": "2025-05-18T12:00:00Z"  
}
```

INVOICE / PDF

GET /api/orders/{id}/invoice – Download invoice PDF

UTILITIES

GET /api/system/languages – Supported locales
GET /api/docs – Swagger API documentation

SECURITY NOTES

All secure endpoints require Authorization: Bearer <JWT>
Admin routes protected via role checks
Rate limiting enabled for login & sensitive routes

Front-End Design Document

Folder Structure

```
src/
├── app/
│   ├── core/           # Singleton services (auth, interceptors)
│   ├── shared/         # Shared modules/components
│   ├── auth/           # Login/Register/OTP
│   ├── dashboard/      # Customer & Admin dashboard
│   ├── products/       # Product catalog
│   ├── orders/         # Order listing & tracking
│   ├── admin/          # Admin controls (stats, product mgmt)
│   ├── invoice/        # PDF invoice rendering
│   ├── app-routing.module.ts
│   └── app.component.ts
```

Component Design

Authentication

LoginComponent
RegisterComponent
VerifyOtpComponent

Product

ProductListComponent
ProductDetailsComponent
ProductFormComponent (Admin)
ProductUploadImageComponent (Admin)

Order

OrderPlacementComponent
MyOrdersComponent
OrderDetailsComponent
TrackOrderComponent (Real-time via WebSocket)

Admin

AdminDashboardComponent
OrderManagementComponent
InventoryAlertComponent

Invoice

InvoiceComponent (Download via API)

Shared

NavbarComponent
FooterComponent
ConfirmDialogComponent
NotificationToastComponent

Routing Design

```

const routes: Routes = [
  { path: '', component: ProductListComponent },
  { path: 'login', component: LoginComponent },
  { path: 'register', component: RegisterComponent },
  { path: 'verify-otp', component: VerifyOtpComponent },
  { path: 'products/:id', component: ProductDetailsComponent },
  {
    path: 'dashboard', component: DashboardComponent,
    children: [
      { path: 'orders', component: MyOrdersComponent },
      { path: 'track/:id', component: TrackOrderComponent },
      { path: 'invoice/:id', component: InvoiceComponent }
    ],
    canActivate: [AuthGuard]
  },
  {
    path: 'admin', component: AdminDashboardComponent,
    children: [
      { path: 'orders', component: OrderManagementComponent },
      { path: 'products', component: ProductFormComponent },
      { path: 'inventory', component: InventoryAlertComponent }
    ],
    canActivate: [AdminGuard]
  }
];

```

Services

Service Responsibilities

AuthService JWT login/logout, token storage, role checks

UserService Fetch/update user profile

ProductService CRUD for products

OrderService Place order, track order, get history

WebSocketService Connect to /ws/orders and listen for updates

NotificationService Toasts & alerts

InvoiceService Download invoice PDF

AdminService Stats, dashboard analytics

Auth & Guards

AuthGuard: protects user dashboard

AdminGuard: allows only Admins

Token stored in localStorage, attached via HTTP interceptor

Optional: refresh token logic

Real-Time Updates

Uses STOMP over WebSocket (@stomp/stompjs, sockjs-client)

Subscribed to /topic/orders/{userId}

Inject WebSocketService into TrackOrderComponent to show status changes

```
this.webSocketService.subscribe(`/topic/orders/${this.userId}`)
```

```
.subscribe((update) => this.updateStatus(update));
```

UI/UX Notes

Responsive design (mobile/tablet/desktop)
Use Angular Material for consistent design
Lazy loading of admin and dashboard modules
Global loading spinner & error toast
Accessible (ARIA attributes)

Testing

Unit Tests: Jasmine/Karma
End-to-End: Cypress or Protractor
Example:

Auth: login success/failure
Products: fetch, filter, CRUD
Orders: create, status update
WebSocket: simulate message push

Environment Setup

```
// environment.ts
export const environment = {
  apiUrl: 'http://localhost:8080/api',
  wsEndpoint: 'http://localhost:8080/ws/orders',
  production: false
};
```

Backend Design Document – Real-Time Order Tracking System

Project Structure

```
src/
├── main/
│   ├── java/com/example/ordertracking/
│   │   ├── config/           # Security, WebSocket, CORS
│   │   ├── controller/      # REST Controllers
│   │   ├── service/         # Business logic
│   │   ├── model/           # Entities
│   │   ├── dto/             # Data Transfer Objects
│   │   ├── repository/      # JPA repositories
│   │   ├── websocket/       # WebSocket config & handlers
│   │   └── util/            # Utilities (JWT, mapping)
│   └── resources/
│       ├── application.yml
│       └── static/templates/ # Optional (invoice templates)
```

Entity Model Overview

User

Long id;
String username;
String email;
String passwordHash;
Role role; // ADMIN, CUSTOMER

Product

Long id;
String name;
String description;
Double price;
Integer stock;
String imageUrl;

Order

Long id;
User user;
LocalDateTime createdAt;
Double totalAmount;
OrderStatus status; // PLACED, PACKED, SHIPPED, DELIVERED

OrderItem

Long id;
Product product;
Order order;
Integer quantity;
Double unitPrice;

DTOs

LoginRequest, RegisterRequest
ProductDTO, OrderDTO, OrderItemDTO
OrderStatusUpdateRequest
InvoiceResponseDTO (for PDF download)

Security Design

Authentication: JWT
Authorization: Role-based
Password Storage: BCrypt

Filters:

JWTFilter: Extract and validate JWT

AuthEntryPoint: Handle unauthorized access

Guards:

@PreAuthorize("hasRole('ADMIN')")

WebSocket Design

Config

@Configuration

@EnableWebSocketMessageBroker

```
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {  
    public void configureMessageBroker(MessageBrokerRegistry config) {  
        config.enableSimpleBroker("/topic");  
        config.setApplicationDestinationPrefixes("/app");  
    }  
}
```

```

    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws/orders").setAllowedOrigins("*").withSockJS();
    }
}

```

OrderStatusPublisher

@Autowired SimpMessagingTemplate messagingTemplate;

```

public void notifyStatusChange(Long orderId, String status, Long userId) {
    messagingTemplate.convertAndSend(
        "/topic/orders/" + userId,
        new OrderStatusUpdate(orderId, status, LocalDateTime.now())
    );
}

```

Service Layer Examples

ProductService

```

public List<ProductDTO> getAllProducts();
public ProductDTO createProduct(ProductDTO productDTO);
public ProductDTO updateProduct(Long id, ProductDTO dto);

```

OrderService

```

public OrderDTO placeOrder(OrderRequest request);
public List<OrderDTO> getOrdersForUser(Long userId);
public void updateOrderStatus(Long orderId, OrderStatus status);

```

REST API Endpoint Overview

Method URL Description

POST /api/auth/login Login with JWT

POST /api/auth/register Register new user

GET /api/products List all products

POST /api/products Create product (Admin only)

PUT /api/products/{id} Update product (Admin only)

DELETE /api/products/{id} Delete product

POST /api/orders Place an order

GET /api/orders/my Get orders for logged-in user

GET /api/orders/{id} Get order by ID

PUT /api/admin/orders/{id}/status Update order status (Admin)

GET /api/admin/stats Get dashboard stats

GET /api/orders/{id}/invoice Download invoice as PDF

Example JWT Auth Flow

User logs in → Receives JWT

JWT sent in Authorization: Bearer <token> on all requests

Interceptor decodes token, validates user/role

Spring Security filters handle role-based checks